

1) Question Set #1 [40 points].

Design decisions: a) What do you consider a token for this task and why (i.e., do you include punctuation marks as well)? What kind of preprocessing steps, if any, do you need to apply before you feed the data into your language model?

I consider tokens in the letter bigram model to be characters in the alphabet, including certain punctuation symbols. I chose tokens to be represented this way because in the letter bigram model, the tokens should be representing units in the alphabet. I decided to preprocess my data by using `lowercase()` to lower all the cases, so that A->a and even the french accented characters can all be matched to the respective lowercase variables. This made my alphabet more concise. I included all alphabet letters "(a..z) and all the types of accented letters ('à',etc). For punctuation, I included (space, <<>>, ' , "" , -) because I know in French they do not use the quotation marks ("") instead they use (<<>>) to denote quotes. French also has a lot of contractions where they combine to words like, for example, (le universit   -> l'universit  ) when the first determiner ends in a vowel and the second noun starts with a vowel, too. So, the corpus would have many (') marks. I also noticed there were lots of (-), so I added it to the alphabet.

b) What technique do you decide to use for out of vocabulary (OOV) words and why?

I did take OOV characters into consideration and I got a 96% accuracy. I assigned OOV characters to "<unk>" in my code. I did see that I got a 96.67% accuracy without taking into consideration any OOVs, though. I think that the number of actual OOV characters is very small, and had little impact on my accuracy. I most likely covered most if not all of the OOV characters in my alphabet, which is why there was little to no change in accuracy with the implementation of OOV characters as <unk>.

c) Can the letter bigram model be implemented without any kind of smoothing? If not, use add-one smoothing. Is this kind of smoothing appropriate or do you need better algorithms? Why (not)?

It can be implemented without any smoothing. I was able to get 284/300 correct lines without Laplace + Log smoothing, but I was able to get 290/300 with the smoothing implementation. I think that there may be a better smoothing algorithm to get closer to 300, though.

2) Question Set #2 [30 points]

Implement a word bigram model, which learns word bigram probabilities from the training data. Again, a separate model will be learned for each language. Design decisions: a) What do you consider as a word for this task and why (i.e., only alpha-numeric characters, or do you want to use punctuation marks as well as valid word tokens)? What kind of preprocessing steps, if any, do you need to apply before you feed the data into your language model?

For this model, I consider words as tokens (words are any combination of characters separated by a space). I do not consider punctuation like “?!,:.” because they are just sentence boundary markers in most cases. I have my sentence boundaries marked as “<s>” and “<e>” in my code. This allows me to take into account what words appear frequently at the start of a sentence and end of sentences. For preprocessing, I use .lowercase() to lower case all of the text. I then remove all of the punctuation with .replace(). I tokenize the sentence with .split(). Then, I add my sentence boundary variables “<s>” and “<e>” to the start and end of the list of tokens. One drawback I noticed is that I am only able to do this because from what I observed of the input data, the lines were representing single sentences for the most part, so each line is a distinct sentence. If there were two distinct lines, my token sentences would have start and end boundaries based on lines of input instead of where the original periods were in the sentences, which would be incorrect, but for this assignment, since the lines are distinct sentences, just adding the start and end boundary symbols to the start and end of my token list would result in a more or less correct representation of the original sentence as tokens.

b) What technique do you decide to use for out of vocabulary (OOV) words and why?

I decided that any OOV words, any words that I did not see in the training data were OOV (unpresent words), thus were assigned “<unk>” in my vocab. I do not consider "rare cases: any word only seen a few times" as OOV in my implementation. I found that if I count rare cases as OOV words, then my accuracy plummets to 3% and below. Thus, I left my unknown variable assignment only to vocabulary that has never been seen before in the training files. This is why my unknowncapacity variable is left to be 0 in my code. It means that any word assigned a count above 0 is a legitimate vocabulary to be added to my vocab set and anything with count 0 is assigned <unk>. I think that since taking rare cases into consideration ruins the accuracy of my code, then keeping the unique words of the three languages is more important for language classification than keeping my vocabulary more compact and stable by assigning rare occurrences to <unk>.

c) Can the word bigram model be implemented without any kind of smoothing? If not, try add-one smoothing. Is this kind of smoothing appropriate or do you need better algorithms? Why (not)? Apply the models to determine the language for each sentence in the test file. Compare your output file with the solution file provided in the src/Data/Validation/folder (labels.sol). How many times was your program correct?

I think this model cannot be efficiently applied without some sort of smoothing technique. I attempted to make it without any smoothing (no Laplace), and I got only 105/300 lines correct compared to 299/300 with Laplace smoothing. This makes sense because the data is finite and there are many combinations of words that may trigger a 0 count in the probability model. This means without smoothing, all of these would be treated as 0 probability and would penalize the score by negative infinity, leading to lower predictability results.

3) Question Set #3 [30 points].

c) What do you do when the number of words seen once are unreliable?

My code implementation uses $N1$ directly, but if $N1$ is 0, I set the unseen probability mass to a very small number ($\text{tinyNum} = 1e-12$), and if no bigram data is available for a given word, the model uses unigram probabilities instead. I was able to get 299/300 without any smoothing or any other techniques to deal with an unreliable $N1$, which means that it was unnecessary to implement a separate case for if the words seen once were unreliable in this assignment, but it may have bigger consequences on other data sets.

Which of the language models at Question Sets #1-3 is the best? Comment on the advantages and disadvantages of these language models on the task (be as detailed as possible based on your observations).

I found the first model, letter model, to be the least reliable. It did not perform too badly overall, but it was outperformed by the word bigram model with Laplace smoothing. The issue with the Letter Bigram Model is that it only looks at the arrangement of letters, so it fails to capture information that could be otherwise captured from grammatical structures, sentence structures, and words in different languages.

I actually found Bigram Laplace to be the most efficient model in this assignment because it takes only 0.2 seconds to run opposed to the Good-Turing smoothing model which took 17.7 seconds to run. It may be that my implementation of Good-Turing smoothing is inefficient, but in this assignment, the Laplace performs just as accurately as Good-Turing with a 10th of the time it takes to run Good-Turing, which could be very useful if the data size was much larger than it is right now. I think it depends on the data size and what types of languages and training sets given for which model is the best. If there was data that was more complex and more unseen words given, then I believe that Good-Turing, with a much bigger test set, would perform more accurately than Laplace. Laplace as a concept is going to inflate 0 probabilities and subtract probabilities from objects with high probability, more than Good-Turing would. Thus, if the set is big enough, Laplace could show more inaccuracies, which means that Good-Turing would be a more accurate approach even if it takes more time to execute. I prefer to have more precise results over run time, but in huge data sets, there are always run time limits.